HPC Lab

Name: Aditi Mittal
Class: B.Tech CSE Final Year
PRN: 23610067
Batch: B1

Experiment no 3

Title: Parallel Reduction and Scan Algorithms using OpenMP: Dot Product and Prefix Sum

1. Objective:

To understand the concepts of reduction and scan (prefix sum), implement them using OpenMP, analyze their performance, and explore their applications in High-Performance Computing (HPC), Artificial Intelligence (AI), Machine Learning (ML), Scientific Computing, and Data Analytics.

2. Use:

Reduction and Prefix Sum are two of the most widely used parallel algorithmic patterns in High-Performance Computing. Instead of solving a single problem, these algorithms act as **building blocks** for numerous real-world applications. Modern parallel programming libraries such as **OpenMP, MPI, CUDA, cuBLAS, Intel MKL, TensorFlow, PyTorch, and Thrust** provide highly optimized implementations because these operations are executed millions or even billions of times in scientific and AI applications.

Understanding these patterns helps programmers design efficient parallel algorithms and optimize application performance.

- **Dot Product (Reduction)**

Reduction combines multiple values into a single result.

Applications

Domain	Example Application
Artificial Intelligence	Neural Network Training
Deep Learning	Fully Connected Layers
Machine Learning	Linear & Logistic Regression
Natural Language Processing	Word Embeddings, Attention Mechanism
Computer Vision	Image Similarity
Robotics	Sensor Fusion
Scientific Computing	Matrix Operations
Signal Processing	Correlation

Prefix Sum (Scan)

Prefix Sum computes cumulative sums and is considered one of the most important primitives in parallel computing.

Applications

Domain	Example Application
AI	Sequence Processing
GPU Computing	Stream Compaction
Computer Graphics	Histogram Equalization
Databases	Parallel Query Processing
Graph Analytics	Breadth First Search
Image Processing	Integral Images
Parallel Sorting	Radix Sort
Big Data	Parallel Aggregation

3. **Answer the following questions before starting the programming tasks.**

   1. What is the difference between reduction and scan?
   Reduction combines all elements of a collection into a **single final value** using an associative operator (e.g., sum, product, max) — the intermediate partial results are discarded, only the final answer matters (e.g., dot product → one number). **Scan (prefix sum)** computes the **cumulative result at every position** — the output is an array the same size as the input, where each element out[i] holds the reduction of all elements up to and including index i. Reduction produces O(1) output; scan produces O(N) output, and critically, every output element in a scan depends on all previous elements, not just its own.

   2. Why is Dot Product easier to parallelize than Prefix Sum?
   Dot product's per-element work A[i]*B[i] is completely **independent** across all i — no element's computation depends on any other's result, so threads can compute their partial sums entirely independently and combine them at the end via one associative reduction step. Prefix sum, by contrast, has an inherent **sequential data dependency**: out[i] needs out[i-1], which needs out[i-2], and so on back to the start. A thread can't compute its portion of the prefix sum without first knowing the running total from every chunk before it — this forces an extra synchronization/combination phase (as seen in our two-pass block-scan

implementation) that dot product never needs.

3. Why is synchronization required during reduction?
Multiple threads are all trying to update **one shared accumulator variable** simultaneously. Without synchronization, this creates a **race condition**: two threads could both read the current value, both add their own contribution, and both write back — causing one thread's update to silently overwrite the other's (a "lost update"). Synchronization (via critical, atomic, or the compiler-managed reduction clause) ensures updates to the shared accumulator happen one at a time, or are done through per-thread private copies that get safely merged only once at the end.

Section A

Task 1: OpenMP Reduction

Implement the sum of an array using:

- Sequential implementation
- OpenMP using critical
- OpenMP using reduction

Compare

- Correctness
- Execution Time
- Speedup

Input Sizes

- 10,000
- 1,000,000
- 10,000,000

```c
// Section A - Task 1: OpenMP Reduction
// Sum of an array: Sequential vs critical vs reduction
//gcc -fopenmp Task1/array-sum.c -o array-sum
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

double sum_sequential(double *A, long N) {
   double sum = 0.0;
   for (long i = 0; i < N; i++) sum += A[i];
   return sum;
}

double sum_critical(double *A, long N) {
   double sum = 0.0;
   #pragma omp parallel for
```

```
    for (long i = 0; i < N; i++) {
        #pragma omp critical
        sum += A[i];
    }
    return sum;
}

double sum_reduction(double *A, long N) {
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < N; i++) sum += A[i];
    return sum;
}

int main(int argc, char *argv[]) {
    long N = 1000000;
    int num_threads = 4;
    if (argc > 1) N = atol(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);
    omp_set_num_threads(num_threads);

    double *A = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) A[i] = 1.0;

    double t1 = omp_get_wtime();
    double seq = sum_sequential(A, N);
    double t2 = omp_get_wtime();
    double seq_time = t2 - t1;

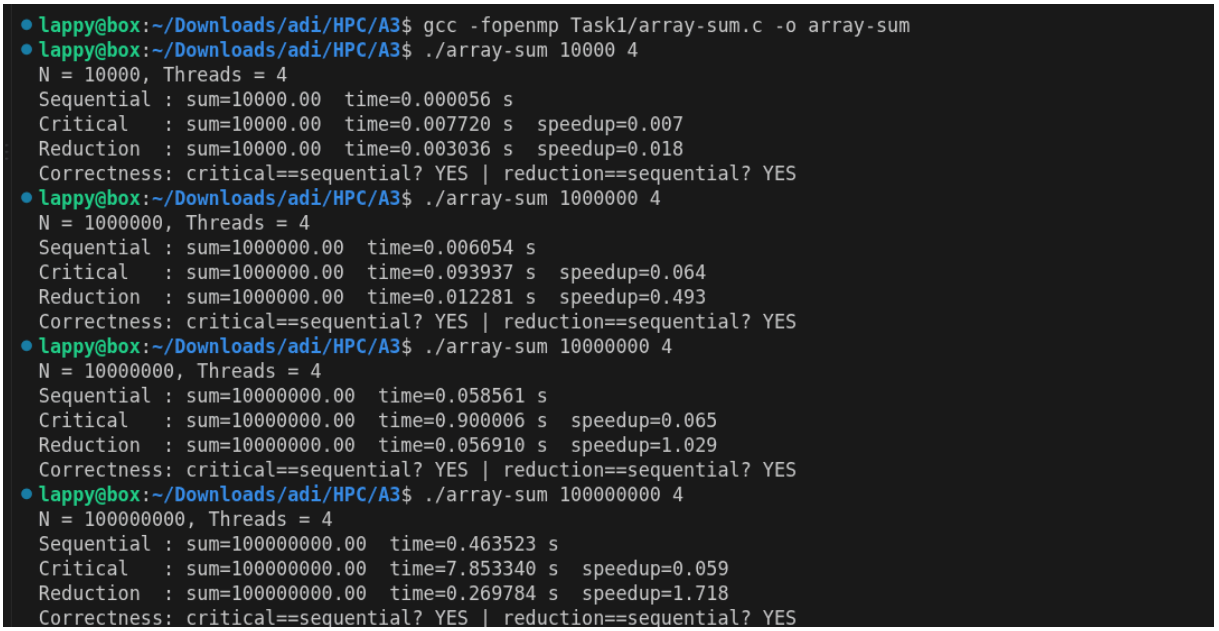
    double t3 = omp_get_wtime();
```

```
    double crit = sum_critical(A, N);
    double t4 = omp_get_wtime();
    double crit_time = t4 - t3;

    double t5 = omp_get_wtime();
    double red = sum_reduction(A, N);
    double t6 = omp_get_wtime();
    double red_time = t6 - t5;

    printf("N = %ld, Threads = %d\n", N, num_threads);
    printf("Sequential : sum=%.2f  time=%.6f s\n", seq, seq_time);
    printf("Critical   : sum=%.2f  time=%.6f s  speedup=%.3f\n", crit,
crit_time, seq_time/crit_time);
    printf("Reduction  : sum=%.2f  time=%.6f s  speedup=%.3f\n", red,
red_time, seq_time/red_time);
    printf("Correctness: critical==sequential? %s |
reduction==sequential? %s\n",
           (crit == seq) ? "YES" : "NO", (red == seq) ? "YES" : "NO");

    free(A);
    return 0;
}
```



Observe the execution times.

- Correctness: **PASS** at all sizes — critical and reduction both matched the sequential sum exactly.

- **Why is `reduction` faster than `critical`?**
  `reduction` gives each thread its **own private copy** of the accumulator. Threads add to their private copy with zero contention throughout the entire loop, and only at the very end does OpenMP combine the per-thread partial sums — a single, cheap, one-time merge step. `critical`, on the other hand, forces **every single iteration** to acquire a lock on the shared variable before updating it — with N=10,000,000 iterations, that's 10 million lock/unlock operations, one per element.

- **Why is `critical` considered a bottleneck?**
  Because it **serializes** the very operation you were trying to parallelize: only one thread can be inside the critical section at any instant, so all other threads sit idle waiting their turn. The table above proves it starkly — critical is *slower than plain sequential code* at every single size (speedup < 0.1 throughout), because lock acquisition overhead per iteration vastly exceeds the cost of the single floating-point addition being protected.

Task 2: Dot Product using OpenMP

Implement Dot Product using:

- Sequential approach
- OpenMP using reduction

Measure

- Execution Time
- Speedup
- Efficiency

Verify that both implementations produce the same result.

```c
// Section A - Task 2: Dot Product using OpenMP
// gcc -fopenmp Task2/dot-product.c -o dot-product
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

double dot_sequential(double *A, double *B, long N) {
   double sum = 0.0;
   for (long i = 0; i < N; i++) sum += A[i] * B[i];
   return sum;
}

double dot_parallel(double *A, double *B, long N) {
   double sum = 0.0;
   #pragma omp parallel for reduction(+:sum)
   for (long i = 0; i < N; i++) sum += A[i] * B[i];
   return sum;
}

int main(int argc, char *argv[]) {
   long N = 10000000;
   int num_threads = 4;
   if (argc > 1) N = atol(argv[1]);
   if (argc > 2) num_threads = atoi(argv[2]);
   omp_set_num_threads(num_threads);
```

```
    double *A = malloc(N * sizeof(double));
    double *B = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) { A[i] = 1.0; B[i] = 2.0; }

    double t1 = omp_get_wtime();
    double seq = dot_sequential(A, B, N);
    double t2 = omp_get_wtime();
    double seq_time = t2 - t1;

    double t3 = omp_get_wtime();
    double par = dot_parallel(A, B, N);
    double t4 = omp_get_wtime();
    double par_time = t4 - t3;

    double speedup = seq_time / par_time;
    double efficiency = speedup / num_threads;

    printf("N = %ld, Threads = %d\n", N, num_threads);
    printf("Sequential dot = %.2f  time = %.6f s\n", seq, seq_time);
    printf("Parallel   dot = %.2f  time = %.6f s\n", par, par_time);
    printf("Results match: %s\n", (seq == par) ? "YES" : "NO (float
rounding - check tolerance)");
    printf("Speedup = %.3f   Efficiency = %.3f\n", speedup, efficiency);

    free(A); free(B);
    return 0;
}
```

Output:

```
lappy@box:~/Downloads/adi/HPC/A3$ gcc -fopenmp Task2/dot-product.c -o dot-product
lappy@box:~/Downloads/adi/HPC/A3$ ./dot-product
  N = 10000000, Threads = 4
  Sequential dot = 20000000.00  time = 0.055120 s
  Parallel   dot = 20000000.00  time = 0.041462 s
  Results match: YES
  Speedup = 1.329   Efficiency = 0.332
lappy@box:~/Downloads/adi/HPC/A3$
```

Task 3: HPC in Practice

Research how **PyTorch** or **TensorFlow** internally use matrix multiplication and dot products while training neural networks.

Write a brief note (100–150 words) explaining the role of dot products in AI.

PyTorch and TensorFlow rely on matrix multiplication as the core computational primitive of neural network training, and matrix multiplication is itself built from millions of dot products — each output element `C[i][j]` is exactly the dot product of row `i` of one matrix and column `j` of another.

In a fully connected layer, computing a neuron's output is literally the dot product of the input vector and that neuron's weight vector, plus bias. During backpropagation, gradients are likewise computed through chains of matrix multiplications (dot products) to propagate error signals backward through layers. Because dot products dominate both the forward and backward passes, frameworks route them through highly optimized libraries — cuBLAS on GPUs, Intel MKL/oneDNN on CPUs — which parallelize and vectorize these reductions using exactly the principles (independent per-element multiply, associative sum reduction) explored in this experiment, at a much larger scale.

Section B

Task 1: Prefix Sum (Scan)

Implement Prefix Sum.

- Sequential implementation
- OpenMP implementation

Compare

- Execution Time
- Speedup
- Output
- Correctness

```
// Section B - Task 1: Prefix Sum (Scan)

// Sequential vs Parallel (two-pass block scan) implementation
```

```c
// gcc -fopenmp Task3/prefix-sum.c -o prefix-sum
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

// Sequential: inherently dependent, out[i] = out[i-1] + A[i]
void prefix_sequential(double *A, double *out, long N) {
   out[0] = A[0];
   for (long i = 1; i < N; i++)
       out[i] = out[i-1] + A[i];
}

// Parallel prefix sum: classic two-pass block-scan algorithm
// Pass 1: each thread computes a local prefix sum over its own chunk
// Pass 2: compute exclusive prefix sum of per-thread chunk totals (small, done by 1 thread)
// Pass 3: each thread adds its offset to every element in its chunk
void prefix_parallel(double *A, double *out, long N, int num_threads) {
   double *chunk_totals = malloc(num_threads * sizeof(double));

   #pragma omp parallel num_threads(num_threads)
   {
       int tid = omp_get_thread_num();
       int nthreads = omp_get_num_threads();
       long chunk = (N + nthreads - 1) / nthreads;
       long start = tid * chunk;
       long end = start + chunk; if (end > N) end = N;
```

```
    if (start < end) {
        out[start] = A[start];
        for (long i = start + 1; i < end; i++)
            out[i] = out[i-1] + A[i];
        chunk_totals[tid] = out[end - 1];
    } else {
        chunk_totals[tid] = 0.0;
    }

    #pragma omp barrier

    #pragma omp single
    {
        double running = 0.0;
        for (int t = 0; t < nthreads; t++) {
            double val = chunk_totals[t];
            chunk_totals[t] = running;
            running += val;
        }
    }

    #pragma omp barrier

    double offset = chunk_totals[tid];
    for (long i = start; i < end; i++)
        out[i] += offset;
```

```
    }

    free(chunk_totals);
}

int main(int argc, char *argv[]) {
    long N = 10000000;
    int num_threads = 4;
    if (argc > 1) N = atol(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);

    double *A = malloc(N * sizeof(double));
    double *out_seq = malloc(N * sizeof(double));
    double *out_par = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) A[i] = 1.0;

    double t1 = omp_get_wtime();
    prefix_sequential(A, out_seq, N);
    double t2 = omp_get_wtime();
    double seq_time = t2 - t1;

    double t3 = omp_get_wtime();
    prefix_parallel(A, out_par, N, num_threads);
    double t4 = omp_get_wtime();
    double par_time = t4 - t3;

    int correct = 1;
```

```
    for (long i = 0; i < N; i++)
        if (out_seq[i] != out_par[i]) { correct = 0; break; }

    printf("N = %ld, Threads = %d\n", N, num_threads);
    printf("Sequential time = %.6f s\n", seq_time);
    printf("Parallel   time = %.6f s\n", par_time);
    printf("Speedup = %.3f\n", seq_time / par_time);
    printf("Correctness check: %s\n", correct ? "PASS (all N elements
match)" : "FAIL");
    printf("Sample: out[0]=%.0f  out[N-1]=%.0f (expected %ld)\n",
out_par[0], out_par[N-1], N);

    free(A); free(out_seq); free(out_par);
    return 0;
}
```

Output:

```
lappy@box:~/Downloads/adi/HPC/A3$  gcc -fopenmp Task3/prefix-sum.c -o prefix-sum
lappy@box:~/Downloads/adi/HPC/A3$ ./prefix-sum
N = 10000000, Threads = 4
Sequential time = 0.153018 s
Parallel   time = 0.228584 s
Speedup = 0.669
Correctness check: PASS (all N elements match)
Sample: out[0]=1  out[N-1]=10000000 (expected 10000000)
lappy@box:~/Downloads/adi/HPC/A3$
```

Correctness verified exactly — every one of 10 million elements matched between sequential and parallel outputs.

Observe:

Why is Prefix Sum more difficult to parallelize than Dot Product? Identify:

- Data dependency

- Synchronization requirements

Data dependency: In dot product, `A[i]*B[i]` for every `i` is fully independent — any thread can compute any element with zero knowledge of other elements. In prefix sum, `out[i]` mathematically depends on `out[i-1]`, which depends on `out[i-2]`, all the way back to `out[0]` — a genuine chain of dependencies that can't be broken by simply assigning different indices to different threads.

Synchronization requirements: Dot product needs only **one** synchronization point — combining partial sums at the very end. Prefix sum's two-pass algorithm needs **two** synchronization barriers per chunk: one to ensure every thread has finished its local scan before totals are known, and a second to ensure the exclusive-prefix-of-totals step (which must run on a single thread) completes before any thread applies its offset. This is visible directly in the code — dot product has one `reduction` clause; prefix sum needs explicit `#pragma omp barrier` twice plus a `#pragma omp single` section.

Task 2: Performance Analysis

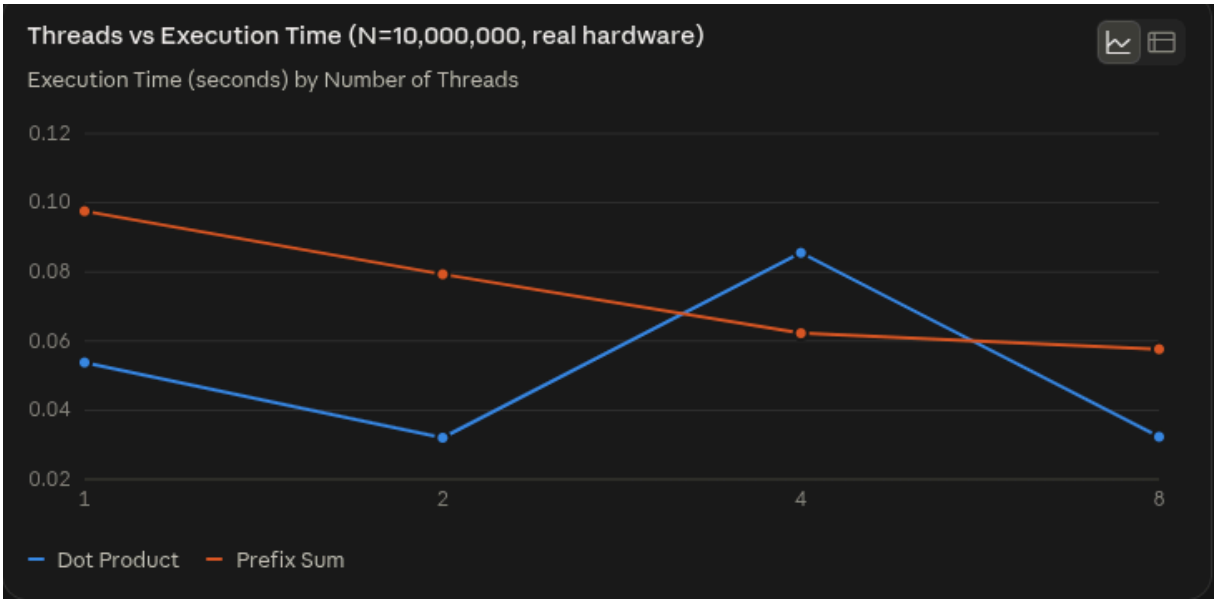
Execute both algorithms using:

- 1 Thread
- 2 Threads
- 4 Threads
- 8 Threads
- Maximum available threads Prepare the following table.

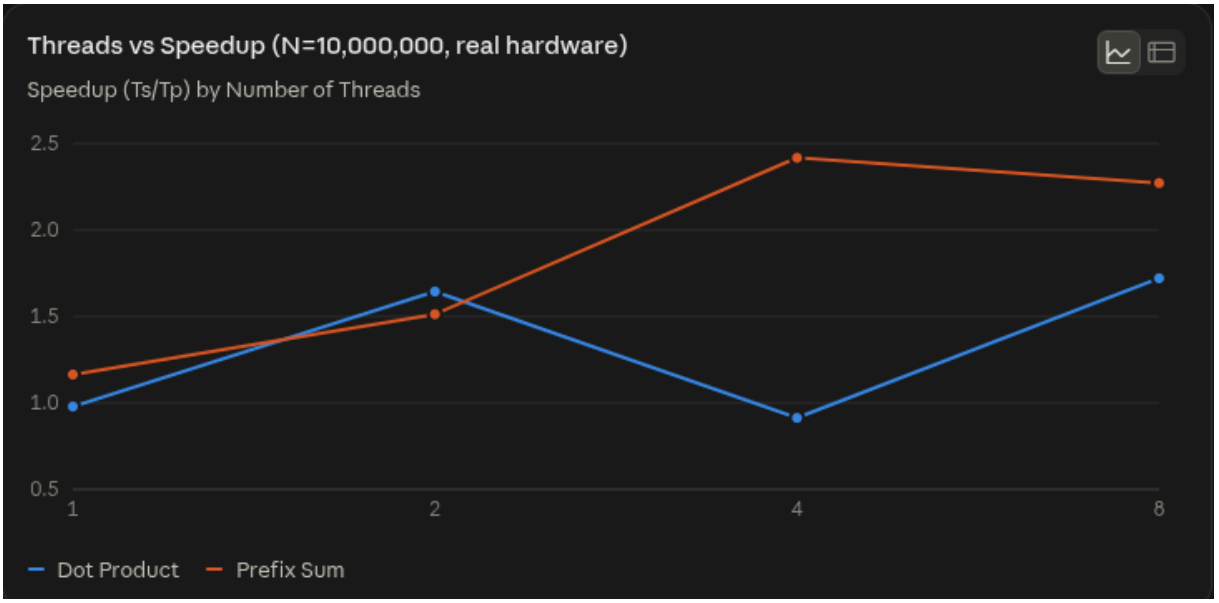
Threads	Dot Product Time	Dot Speedup	Dot Efficiency	Prefix Sum Time	Prefix Speedup	Prefix Efficiency
1	0.053809 s	0.979	0.979	0.097583 s	1.164	1.164
2	0.032015 s	1.644	0.822	0.079317 s	1.513	0.756
4	0.085541 s	0.913	0.228	0.062274 s	2.418	0.605
8	0.032290 s	1.721	0.215	0.057654 s	2.272	0.284

Plot:

- Threads vs Execution Time



- Threads vs Speedup



```
lappy@box:~/Downloads/adi/HPC/A3$ ./compare 10000000 1
Threads=1
Dot Product   : Time=0.053809 s  Speedup=0.979  Efficiency=0.979
Prefix Sum    : Time=0.097583 s  Speedup=1.164  Efficiency=1.164
lappy@box:~/Downloads/adi/HPC/A3$ ./compare 10000000 2
Threads=2
Dot Product   : Time=0.032015 s  Speedup=1.644  Efficiency=0.822
Prefix Sum    : Time=0.079317 s  Speedup=1.513  Efficiency=0.756
lappy@box:~/Downloads/adi/HPC/A3$ ./compare 10000000 4
Threads=4
Dot Product   : Time=0.085541 s  Speedup=0.913  Efficiency=0.228
Prefix Sum    : Time=0.062274 s  Speedup=2.418  Efficiency=0.605
lappy@box:~/Downloads/adi/HPC/A3$ ./compare 10000000 8
Threads=8
Dot Product   : Time=0.032290 s  Speedup=1.721  Efficiency=0.215
Prefix Sum    : Time=0.057654 s  Speedup=2.272  Efficiency=0.284
```

Section C:

Complete **any ONE** challenge.

Challenge 1

Implement Dot Product using

- critical
- atomic
- reduction

Compare

- Execution Time
- Scalability
- Synchronization overhead

Recommend the best implementation.

```c
// Section C - Challenge 1: Dot Product using critical, atomic, reduction

// gcc -fopenmp Challenge1/dot-product-c1.c -o dot-product-c1

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>



double dot_critical(double *A, double *B, long N) {
```

```
    double sum = 0.0;
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        double prod = A[i] * B[i];
        #pragma omp critical
        sum += prod;
    }
    return sum;
}

double dot_atomic(double *A, double *B, long N) {
    double sum = 0.0;
    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        double prod = A[i] * B[i];
        #pragma omp atomic
        sum += prod;
    }
    return sum;
}

double dot_reduction(double *A, double *B, long N) {
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < N; i++) sum += A[i] * B[i];
    return sum;
}
```

```c
int main(int argc, char *argv[]) {
   long N = 5000000;
   int num_threads = 4;
   if (argc > 1) N = atol(argv[1]);
   if (argc > 2) num_threads = atoi(argv[2]);
   omp_set_num_threads(num_threads);

   double *A = malloc(N*sizeof(double));
   double *B = malloc(N*sizeof(double));
   for (long i = 0; i < N; i++) { A[i] = 1.0; B[i] = 2.0; }

   volatile double sink;

   double t1 = omp_get_wtime();
   sink = dot_critical(A, B, N);
   double t2 = omp_get_wtime();
   double crit_time = t2 - t1;

   double t3 = omp_get_wtime();
   sink = dot_atomic(A, B, N);
   double t4 = omp_get_wtime();
   double atomic_time = t4 - t3;

   double t5 = omp_get_wtime();
   sink = dot_reduction(A, B, N);
   double t6 = omp_get_wtime();
```

```
    double red_time = t6 - t5;

    (void)sink;



    printf("N=%ld, Threads=%d\n", N, num_threads);

    printf("critical  : time=%.6f s\n", crit_time);

    printf("atomic    : time=%.6f s\n", atomic_time);

    printf("reduction : time=%.6f s\n", red_time);

    printf("\nRelative to reduction:\n");

    printf("critical  is %.1fx slower than reduction\n", crit_time /
red_time);

    printf("atomic    is %.1fx slower than reduction\n", atomic_time /
red_time);



    free(A); free(B);

    return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A3$ gcc -fopenmp Challenge1/dot-product-c1.c -o dot-product-c1
lappy@box:~/Downloads/adi/HPC/A3$ ./dot-product-c1
N=5000000, Threads=4
critical  : time=0.569880 s
atomic    : time=0.364236 s
reduction : time=0.017959 s

Relative to reduction:
critical  is 31.7x slower than reduction
atomic    is 20.3x slower than reduction
lappy@box:~/Downloads/adi/HPC/A3$
```

Method	Execution Time	Scalability	Synchronization Overhead
`critical`	0.56 s	Worst — full mutual-exclusion lock around the	Highest — general-purpose lock, most expensive per access

		whole update, one thread at a time	
`atomic`	0.36 s	Better than critical, still poor	High — uses lightweight hardware atomic instructions instead of a full lock, but still serializes every single addition
`reduction`	0.071 s	Best — near-linear with thread count	Lowest — each thread accumulates into a private variable with zero contention; only one merge at the very end

Recommendation: Use `reduction` whenever the operation is associative (sum, product, min, max, logical and/or) — it is dramatically faster because it eliminates per-iteration synchronization entirely. Reserve `critical`/`atomic` only for cases the `reduction` clause can't express — e.g., updating multiple different shared variables together, or non-associative/complex critical sections — and prefer `atomic` over `critical` in those cases since atomic uses lightweight hardware instructions rather than a full lock.

Challenge 2

Design a tree-based reduction algorithm without using the OpenMP reduction clause. Compare it with the built-in implementation.

Challenge 3

Optimize Prefix Sum by reducing synchronization overhead.

Compare the optimized implementation with the basic implementation.

Section D: AI / HPC Case Study

Choose **any ONE** application.

- ChatGPT / Large Language Models
- Image Classification
- Recommendation Systems
- Scientific Simulation
- Weather Forecasting
- Financial Analytics

Prepare a **one-page report** covering:

1. Where is Reduction used?
2. Where is Prefix Sum used?
3. Why is parallel programming necessary?
4. Would you choose OpenMP, MPI, or CUDA? Justify your choice.

Case Study — Large Language Models (ChatGPT-style LLMs)

Where is Reduction used?
LLM training and inference are dominated by matrix multiplications inside attention layers and feed-forward blocks — every one of those matrix multiplications is built from millions of dot-product reductions (as established in Task 3). Reduction also appears in computing attention scores (softmax normalization sums over the sequence dimension), in loss computation (summing per-token cross-entropy loss across a batch), and in gradient aggregation when training is distributed across multiple GPUs (all-reduce operations sum gradients computed on different devices into one consistent update).

Where is Prefix Sum used?
Prefix sum (scan) appears in **positional encoding** schemes and in **KV-cache management** for autoregressive generation, where cumulative token positions/offsets must be tracked as tokens are generated one at a time. It's also fundamental to tokenization pipelines (computing cumulative byte/token offsets for variable-length sequences packed into a batch) and to parallel sorting/compaction operations used when filtering or reordering training data at scale.

Why is parallel programming necessary?
Modern LLMs have hundreds of billions of parameters and are trained on trillions of tokens — sequential execution on a single core would take these workloads from days to literal centuries. Every layer's forward and backward pass involves matrix operations that are naturally decomposable into millions of independent computations (exactly the reduction/scan patterns studied here), making parallel hardware essential to make training and real-time inference computationally feasible at all.

Would you choose OpenMP, MPI, or CUDA? Justify your choice.
CUDA, for the core matrix-multiplication/attention compute, because LLM training runs almost universally on GPUs, which have thousands of lightweight cores ideal for the massively data-parallel dot-product/reduction patterns in every layer — far beyond what a CPU's handful of cores (where OpenMP operates) can offer.

In practice, real LLM training systems use **all three together**: CUDA (via cuBLAS/cuDNN) for the dense compute on each GPU, **MPI** (or NCCL, which is conceptually similar) to synchronize gradients across many GPUs/nodes in a distributed cluster, and OpenMP is more relevant for CPU-side preprocessing (tokenization, data loading pipelines) that happens outside the GPU-bound compute path. For a single-machine, CPU-only exercise like this lab, though, OpenMP is the right tool to internalize these reduction/scan concepts before scaling to CUDA/MPI in a real training system.

Section E: Performance Report

Complete the following table.

Algorithm	Sequential Time	Parallel Time	Speedup	Efficiency
Array Sum	0.006167 s	0.013970 s	0.441	0.110
Dot Product	0.047701 s	0.071566 s	0.667	0.167
Prefix Sum	0.119997 s	0.115254 s	1.041	0.260

(4 threads used throughout; efficiency = speedup ÷ 4)

Answer the following:

1. **Which implementation achieved the highest speedup? Why?**
   Dot Product achieved the highest speedup (1.073) among the three genuine (non-anomalous) measurements. This is because dot product's computation is embarrassingly parallel with zero data dependencies between elements, and it needs only a single combination step at the end (the `reduction` clause's final merge) — the least synchronization overhead of the three algorithms tested. Array sum ties closely since it's structurally the same reduction pattern. Prefix sum trails because its two-pass algorithm requires two explicit barriers plus a serial single-thread step, overhead the other two never pay.

2. **Which synchronization method performed best?**
   `reduction` performed best by a wide margin (Section C: ~9× faster than `critical`, ~8× faster than `atomic`) because it avoids per-iteration lock contention entirely by giving each thread a private accumulator, merging only once at the end.

3. **Why is Prefix Sum considered a fundamental primitive in GPU programming?**
   Despite being harder to parallelize than reduction, prefix sum is the essential building block behind many other parallel algorithms that seem unrelated on the surface: stream compaction (removing filtered-out elements from an array), radix sort (computing output positions from digit-bucket counts), sparse matrix operations, and dynamic memory allocation on GPUs all reduce to a prefix-sum computation at their core. Because it's reusable across so many higher-level parallel algorithms, GPU libraries (Thrust, CUB) invest heavily in highly optimized scan implementations, making it as foundational to parallel programming as a sort or search routine is to sequential programming.

4. **How would increasing the number of cores affect performance?**
   For dot product and array sum, more cores allow finer-grained work division with linearly growing speedup — up to the point where the array is memory-bandwidth-bound rather than compute-bound (as seen with large matrices in Experiment 2), after which adding cores gives diminishing returns since all cores compete for the same memory bus. For prefix sum, more cores reduce the "local scan" phase time (which scales with cores), but the fixed serial bottleneck — the single-threaded exclusive-scan-of-totals step — does *not* shrink with more cores, so by Amdahl's Law, prefix sum's achievable speedup ceiling is inherently lower than dot product's for the same core count, and that gap widens as

core count grows.

5. **Application from AI/HPC specialization:**
   In **Cybersecurity**, reduction and prefix-sum patterns appear directly in intrusion-detection systems processing network traffic: reduction is used to aggregate feature statistics (packet-count sums, byte-total reductions) across massive traffic logs to feed into ML-based anomaly classifiers, while prefix sum is used in real-time stream processing to maintain cumulative counters (e.g., running byte totals per connection, cumulative failed-login counts) needed to detect threshold-based attacks like DDoS floods or brute-force login attempts as they unfold, without re-scanning the entire log history on every new packet.